

MPI Collective Communications - Algorithms and Communication Times

Lecture 16

March 16, 2026

Communication Cost Model

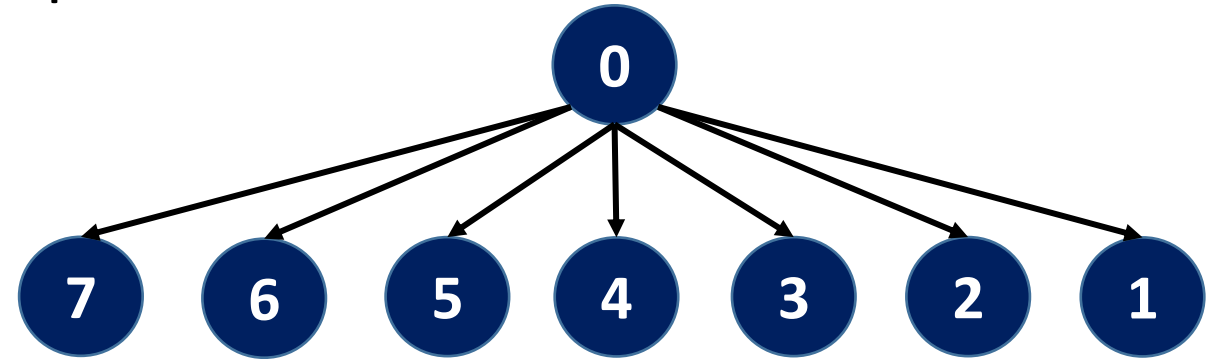
- Communication (or message transfer) time is modeled as $L+n/B$, where L is the latency (or startup time) per message, and $1/B$ is the transfer time per byte, and n is the message size in bytes

Optimization of Collective Communication Operations in MPICH

Rajeev Thakur, Rolf Rabenseifner, William Gropp
IJHPCA 2005

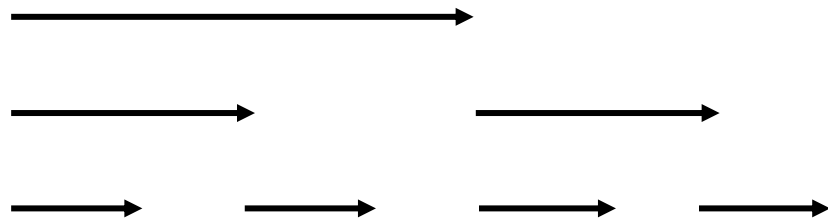
Recap: Broadcast – Naïve Algorithm

- Root process sends to every other process

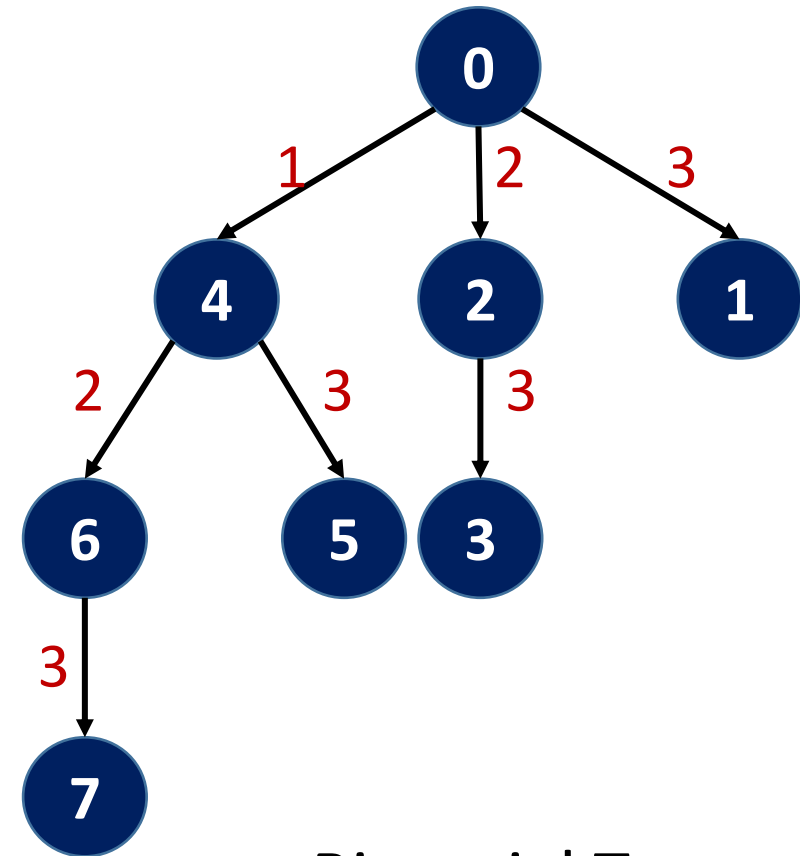


- #Steps for $p (=2^d)$ processes?
 - $p-1$
- Communication time for n bytes
 - $T(p) = (p - 1) * (L + n/B)$
 - $T(p^2) = (p^2 - 1) * (L + n/B)$

Recap: MPI_Bcast – Binomial Algorithm



- #Steps for $p (=2^d)$ processes?
 - $\log p$
- Communication time for n bytes
 - $T(p) = \log p * (L + n/B)$
 - $T(p^2) = 2 \log p * (L + n/B)$



Binomial Tree

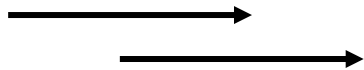
Code

https://github.com/pmodels/mpich/blob/81da5f5bb2dcd33a733083f284a030935eb709ac/src/mpi/coll/bcast/bcast_intra_binomial.c#L61

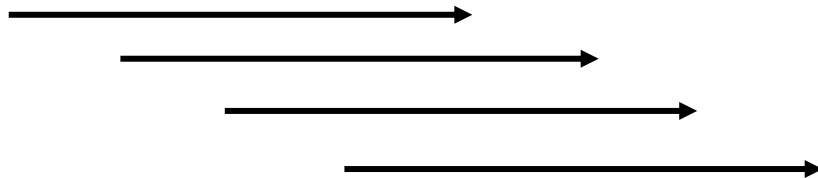
MPI_Bcast – Modified Pattern



Communication step 1



Communication step 2

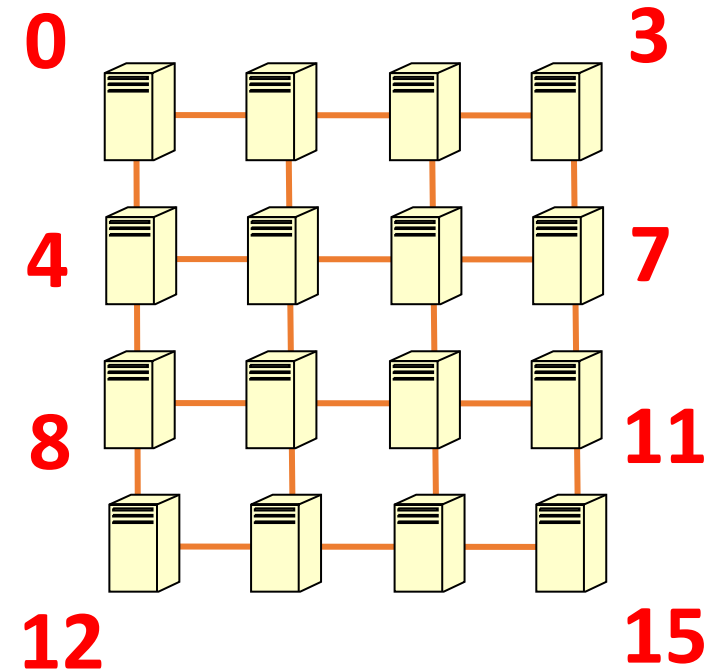


Communication step 3

Recap: MPI_Bcast Communications on 2D Mesh

#1	0 -> 8 (2)	
#2	0 -> 4 (1)	8 -> 12 (1)
#3	0 -> 2 (2)	4 -> 6 (2)
	8 -> 10 (2)	12 -> 14 (2)
#4	0 -> 1 (1)	8 -> 9 (1)
	2 -> 3 (1)	10 -> 11 (1)
	4 -> 5 (1)	12 -> 13 (1)
	6 -> 7 (1)	14 -> 15 (1)

Total #hops for the above: 2 + 1 + 2 + 1
(considering maximum #hops per round)

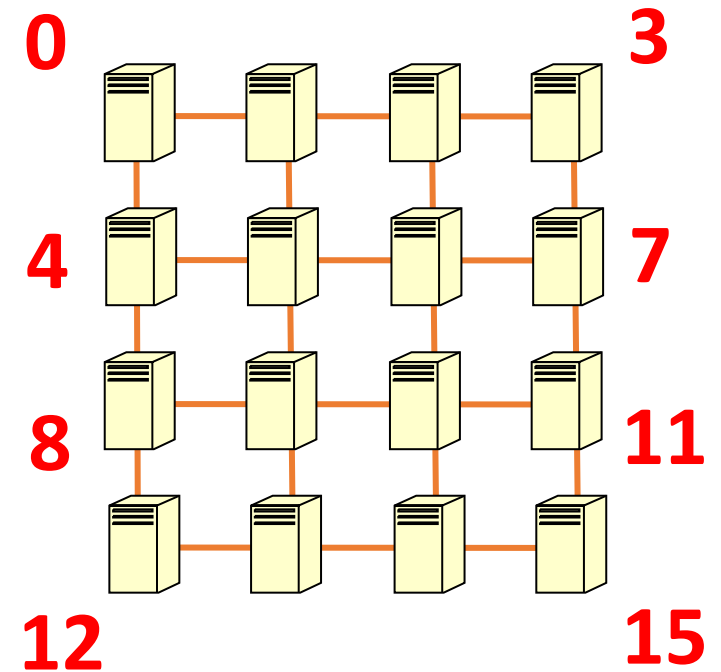


Recap: higher #hops indicate higher communication times

MPI_Bcast Communications on 2D Mesh – Binomial ($np=16$, $ppn=1$)

#1	0 -> 8 (2)	
#2	0 -> 4 (1)	8 -> 12 (1)
#3	0 -> 2 (2)	4 -> 6 (2)
	8 -> 10 (2)	12 -> 14 (2)
#4	0 -> 1 (1)	8 -> 9 (1)
	2 -> 3 (1)	10 -> 11 (1)
	4 -> 5 (1)	12 -> 13 (1)
	6 -> 7 (1)	14 -> 15 (1)

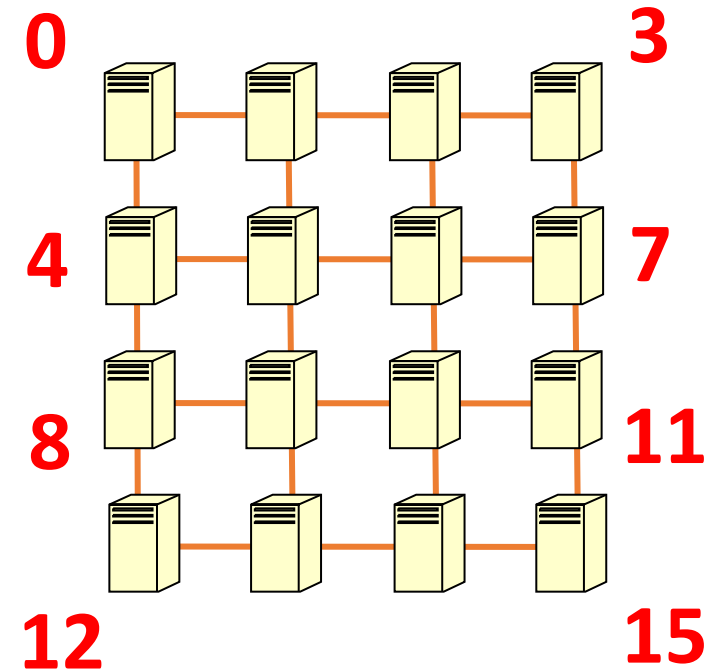
Green implies non-overlapping paths,
thus no congestion



*Recap: higher #hops indicate
higher communication times*

MPI_Bcast Communications on 2D Mesh – Modified pattern (np=16, ppn=1)

#1	0 -> 1 (1)	
#2	0 -> 2 (2)	1 -> 3 (2)
#3	0 -> 4 (1)	1 -> 5 (1)
	2 -> 6 (1)	3 -> 7 (1)
#4	0 -> 8 (2)	1 -> 9 (2)
	2 -> 10 (2)	3 -> 11 (2)
	4 -> 12 (2)	5 -> 13 (2)
	6 -> 14 (2)	7 -> 15 (2)



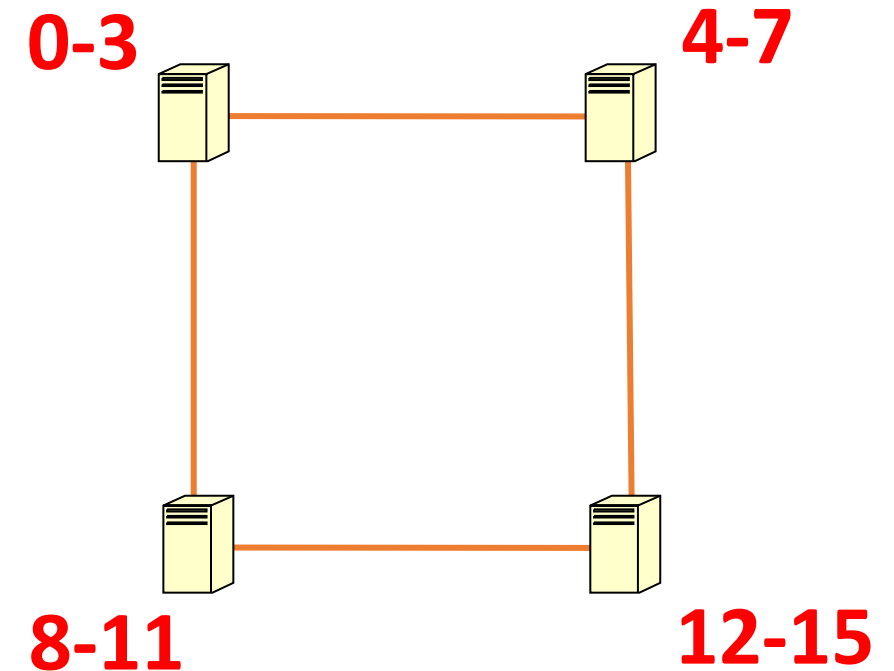
Total #hops for the above: 1 + 2 + 1 + 2

Red implies overlapping paths, hence congestion on these links

MPI_Bcast Communications on 2D Mesh - Binomial (np=16, ppn=4)

#1	0 -> 8 (1)	
#2	0 -> 4 (1)	8 -> 12 (1)
#3	0 -> 2 (0)	4 -> 6 (0)
	8 -> 10 (0)	12 -> 14 (0)
#4	0 -> 1 (0)	8 -> 9 (0)
	2 -> 3 (0)	10 -> 11 (0)
	4 -> 5 (0)	12 -> 13 (0)
	6 -> 7 (0)	14 -> 15 (0)

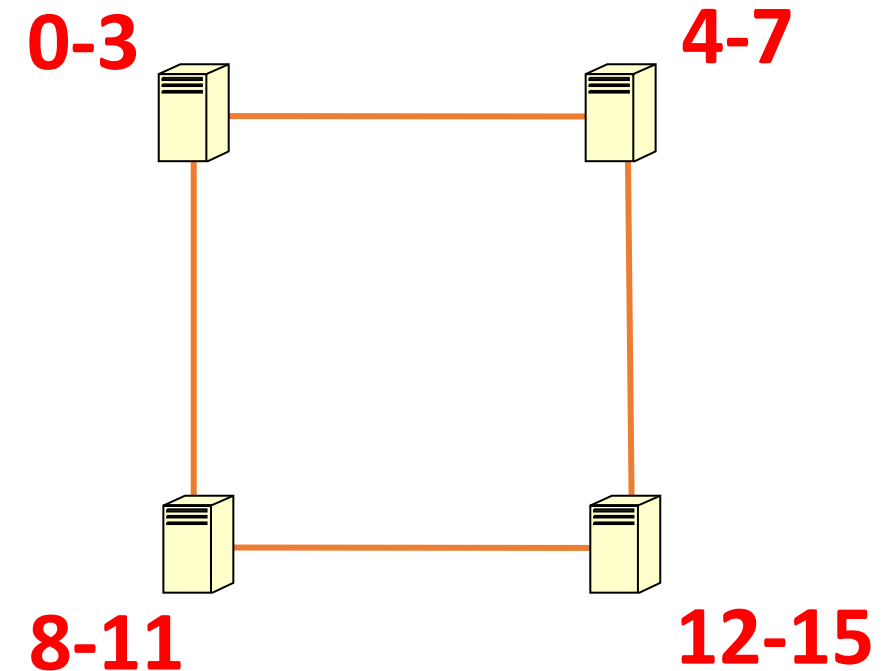
Total #hops for the above: $1 + 1 + 0 + 0 = 2$
(considering maximum #hops per round)



MPI_Bcast Communications on 2D Mesh – Modified pattern (np=16, ppn=4)

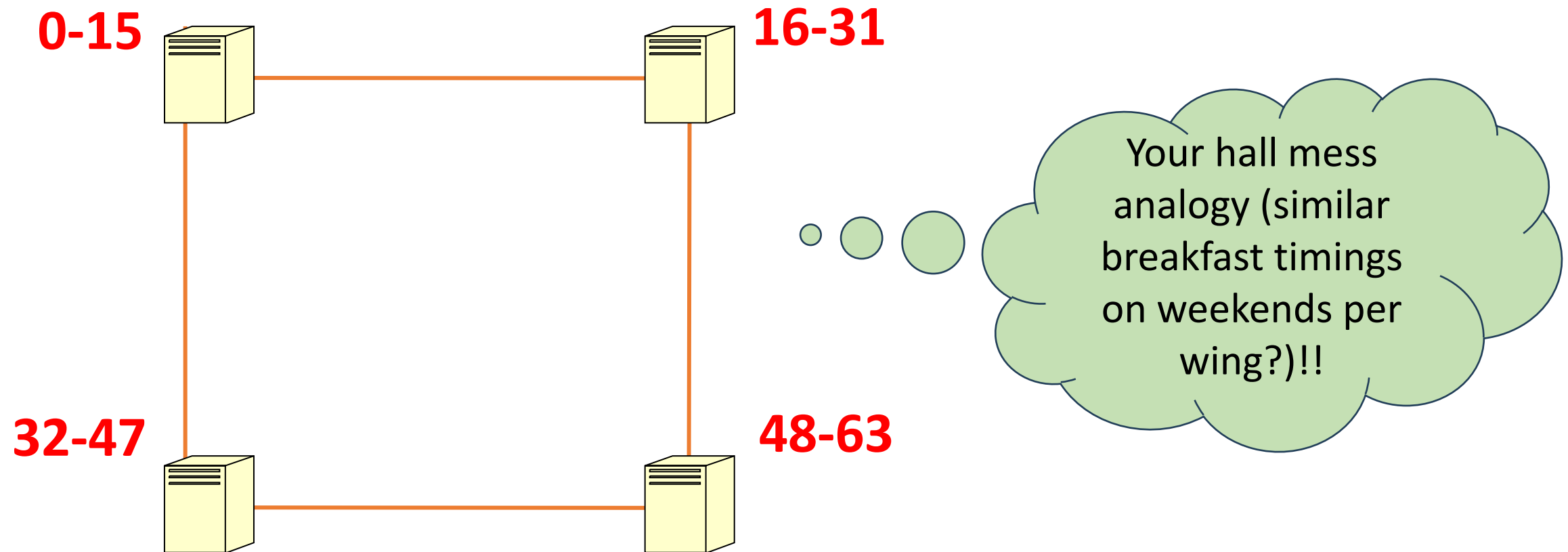
#1	0 -> 1 (0)	
#2	0 -> 2 (0)	1 -> 3 (0)
#3	0 -> 4 (1)	1 -> 5 (1)
	2 -> 6 (1)	3 -> 7 (1)
#4	0 -> 8 (1)	1 -> 9 (1)
	2 -> 10 (1)	3 -> 11 (1)
	4 -> 12 (1)	5 -> 13 (1)
	6 -> 14 (1)	7 -> 15 (1)

Total #hops for the above: 0 + 0 + 1 + 1 = 2
However note that most communications
(steps #2, #3 and #4) may overlap

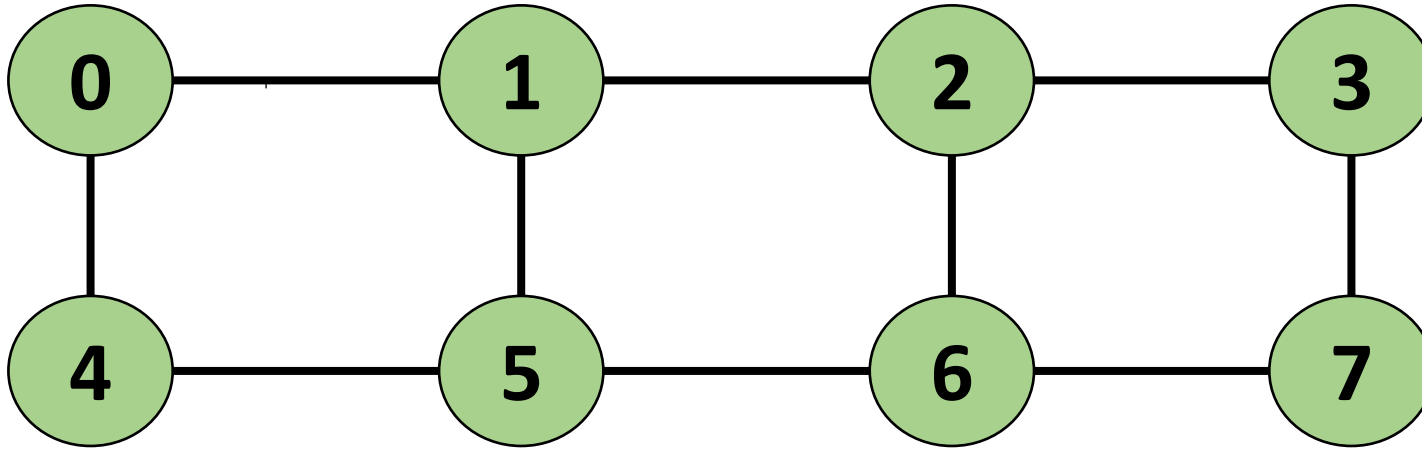


Note: intra-node communication times are non-zero

MPI_Bcast Communications on 2D Mesh – Binomial vs. Modified pattern (np=64, ppn=16)



MPI_Bcast Binomial Communication Cost



0 -> 4

0 -> 2, 4 -> 6

0 -> 1, 2 -> 3

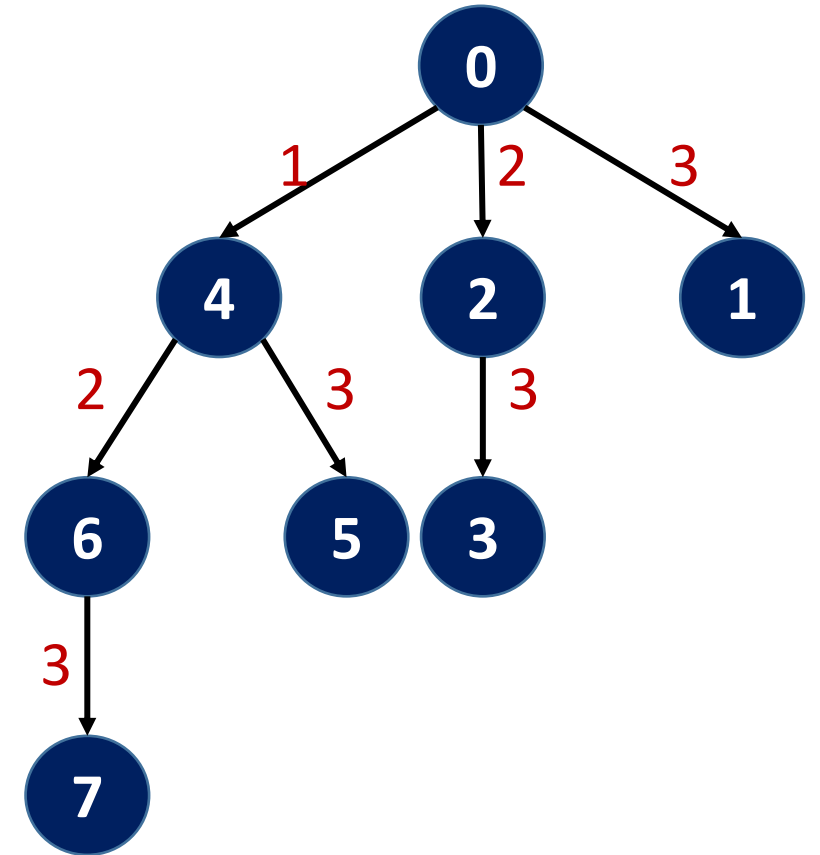
4 -> 5, 6 -> 7

$L + n/B$

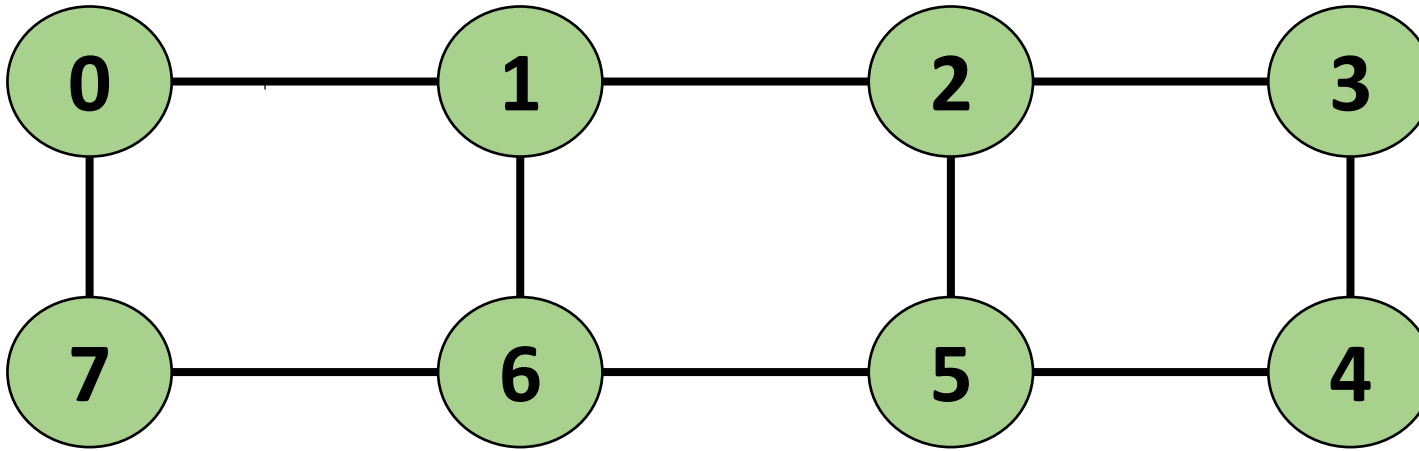
$2L + n/B$

$L + n/B$

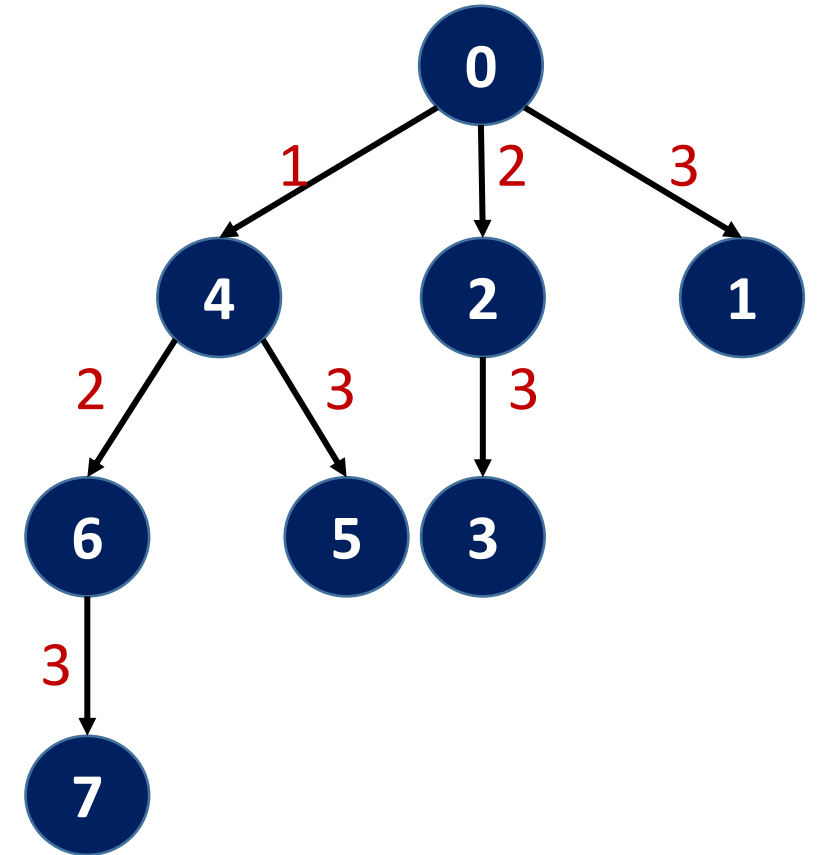
$4L + (\log_2 8) n/B$



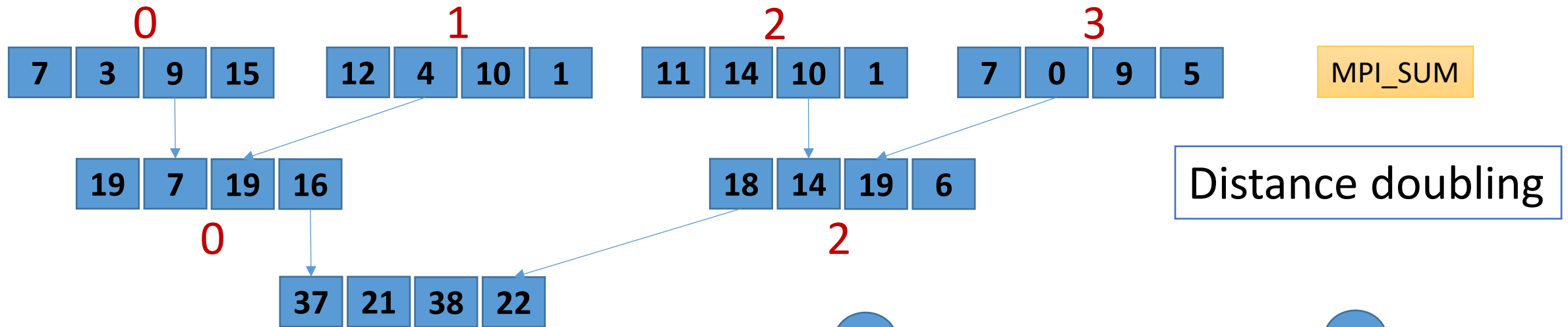
MPI_Bcast Binomial Communication Cost



$$7L + (\log_2 8) n/B$$

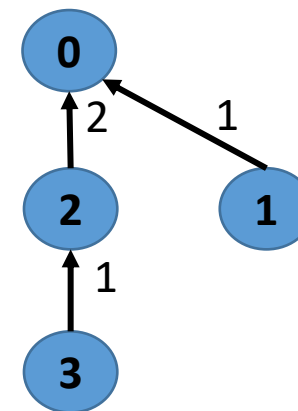


Recap: Reduce Recursive Doubling Algorithm

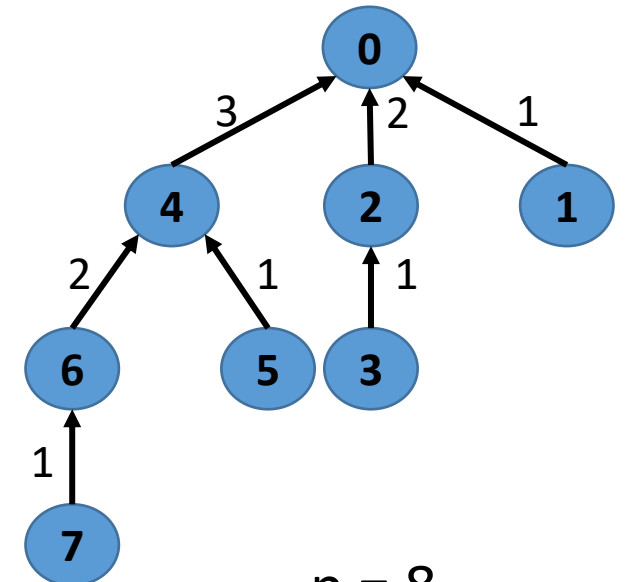


Time: $\log p (L + n \cdot (1/B) + n \cdot c)$
 L = latency, B = bandwidth
 c = compute cost per byte

Used for short messages

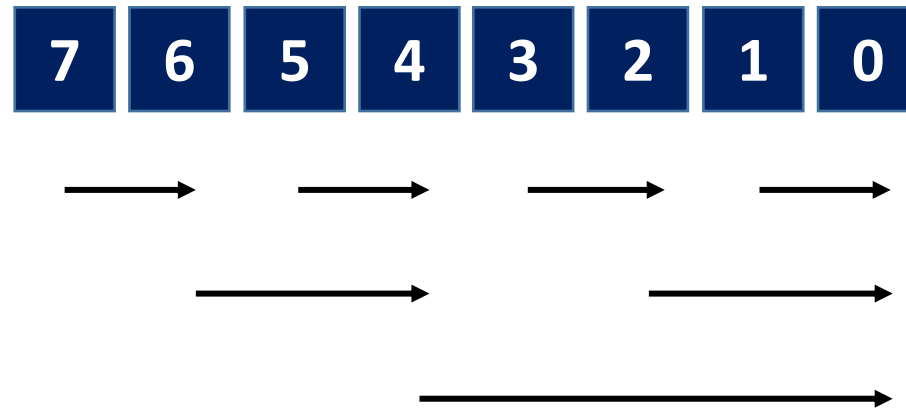


$p = 4$



$p = 8$

MPI_Gather Recursive Doubling Algorithm



Vector doubling
Distance doubling

Message size at every process = n

Every step the message size doubles: $n \cdot 2^0, n \cdot 2^1, \dots, n \cdot 2^{((\log p)-1)}$

Time for scatter of n bytes from root

$$\log p * L + (p-1)*(n)*(1/B)$$

$$\log p * L + (p-1)*(n/p)*(1/B) \quad [\text{Message size} = n/p \text{ at every process}]$$

Hop-bytes

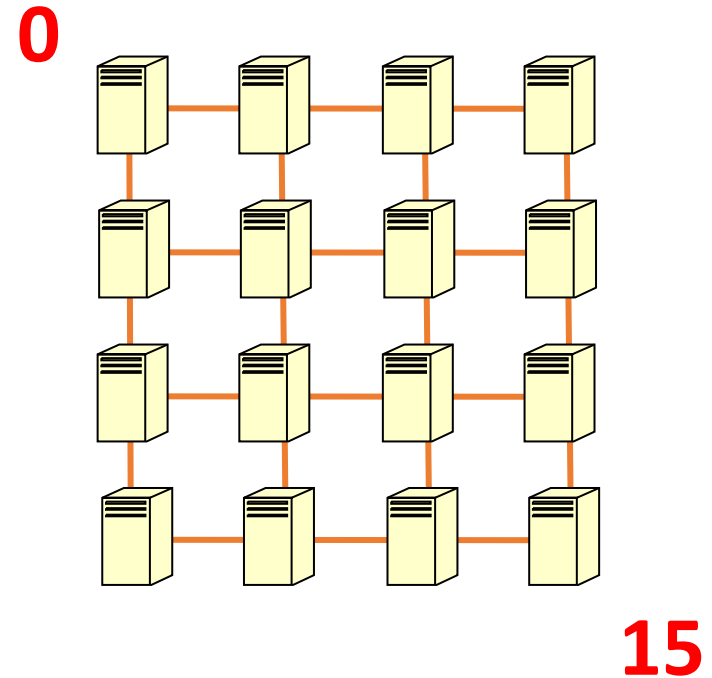
- **Hop-bytes (Metric)** Hop-bytes is the metric (or evaluation function) used to judge the quality of the solution produced by the mapping algorithm. Hop-bytes is the total size of inter-processor communication in bytes weighted by distance between the respective end-processors. The relevant measure for distance between two processors is the length of the shortest path between them in the topology-graph.

Tarun Agarwal et al., “Mapping heuristics for reducing communication cost in parallel machines” (part of MS Thesis, UIUC, 2005)

<http://charm.cs.uiuc.edu/users/amits/topolbpaper.pdf>

MPI_Gather Communications (Hop-bytes)

#1	$0 \leftarrow 1 (1, n)$	$8 \leftarrow 9 (1, n)$
	$2 \leftarrow 3 (1, n)$	$10 \leftarrow 11 (1, n)$
	$4 \leftarrow 5 (1, n)$	$12 \leftarrow 13 (1, n)$
	$6 \leftarrow 7 (1, n)$	$14 \leftarrow 15 (1, n)$
#2	$0 \leftarrow 2 (2, 2n)$	$4 \leftarrow 6 (2, 2n)$
	$8 \leftarrow 10 (2, 2n)$	$12 \leftarrow 14 (2, 2n)$
#3	$0 \leftarrow 4 (1, 4n)$	$8 \leftarrow 12 (1, 4n)$
#4	$0 \leftarrow 8 (2, 8n)$	



$$\text{Total hop-bytes} = 1*n + 2*2n + 1*4n + 2*8n = 25n$$

MPI_Gather Timing

```
CS633 $ time mpirun -np 12 --hosts csews98:6,csews99:6  
./gather 10000000
```

4.54 s

Assume a tree network
connecting the three hosts

```
CS633 $ time mpirun -np 12 --hosts  
csews98:4,csews99:4,csews100:4 ./gather 10000000
```

5.98 s

MPI_Gather Timing

```
CS633 $ time mpirun -np 12 --hosts csews98:6,csews99:6 ./gather 100
```

0.18 s

```
CS633 $ time mpirun -np 12 --hosts csews98:4,csews99:4,csews100:4 ./gather  
100
```

0.19 s

MPI_Gather – Scaling (Constant Total Data Size/Communication Volume)

```
CS633 $ time mpirun -np 24 --hosts csews98:12,csews99:12 ./gather 12000000  
10.81 s
```

```
CS633 $ time mpirun -np 12 --hosts csews98:6,csews99:6 ./gather 24000000  
10.77 s
```

```
CS633 $ time mpirun -np 6 --hosts csews98:3,csews99:3 ./gather 48000000  
10.94 s
```

Throughput =

$12000000 * 24 * 8 \text{ bytes} / 10.81 \text{ sec} = 203.45 \text{ MB/s}$ (assuming 1 double = 8 bytes)

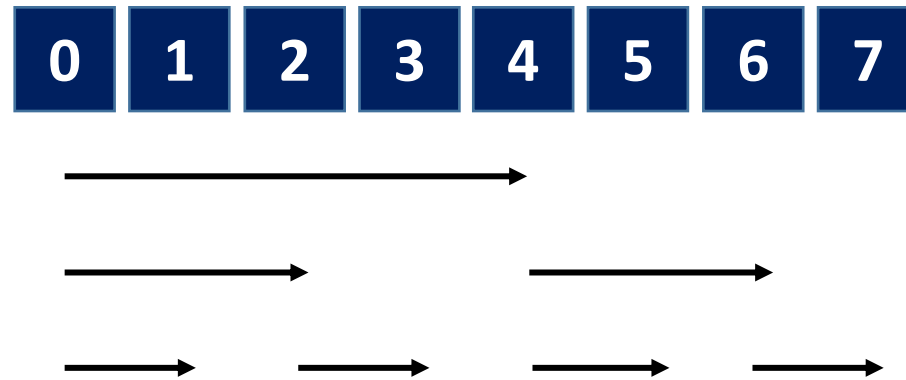
MPI_Gather – Scaling (Total communication volume reduced in proportion to the process count)

CS633 \$ time mpirun -np 24 --hosts csews98:12,csews99:12 ./gather 12000000
10.659s *Total communication volume = 12000000*24*8 bytes*

CS633 \$ time mpirun -np 12 --hosts csews98:6,csews99:6 ./gather 12000000
5.430s *Total communication volume = 12000000*12*8 bytes*

CS633 \$ time mpirun -np 6 --hosts csews98:3,csews99:3 ./gather 12000000
2.864s *Total communication volume = 12000000*6*8 bytes*

MPI_Scatter



Vector halving
Distance halving

Message size at root = n

Every step the message size halves: $n/2, n/2^2, \dots, n/2^{(\log p)}$

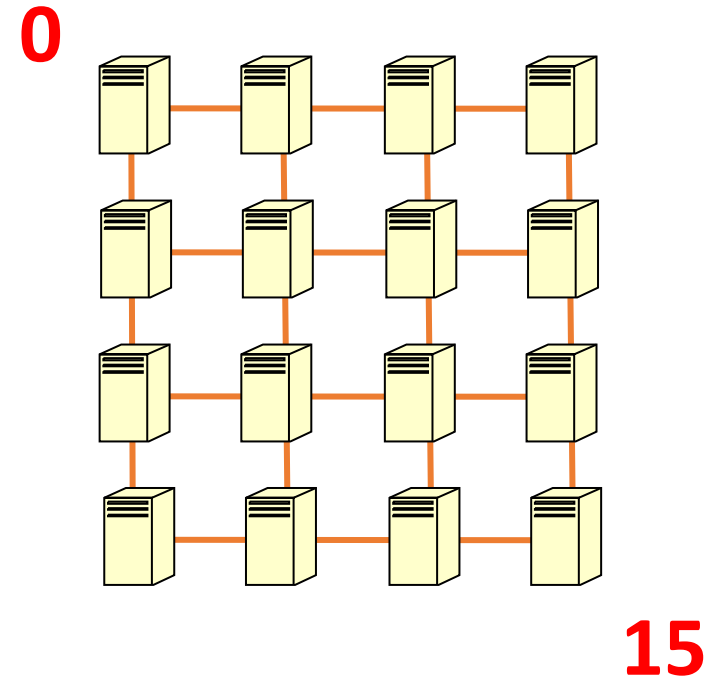
Time for scatter of n bytes from root

$$\log p * L + (p-1)*(n/p)*(1/B)$$

MPI_Scatter Communications

Message size at root = $16n$

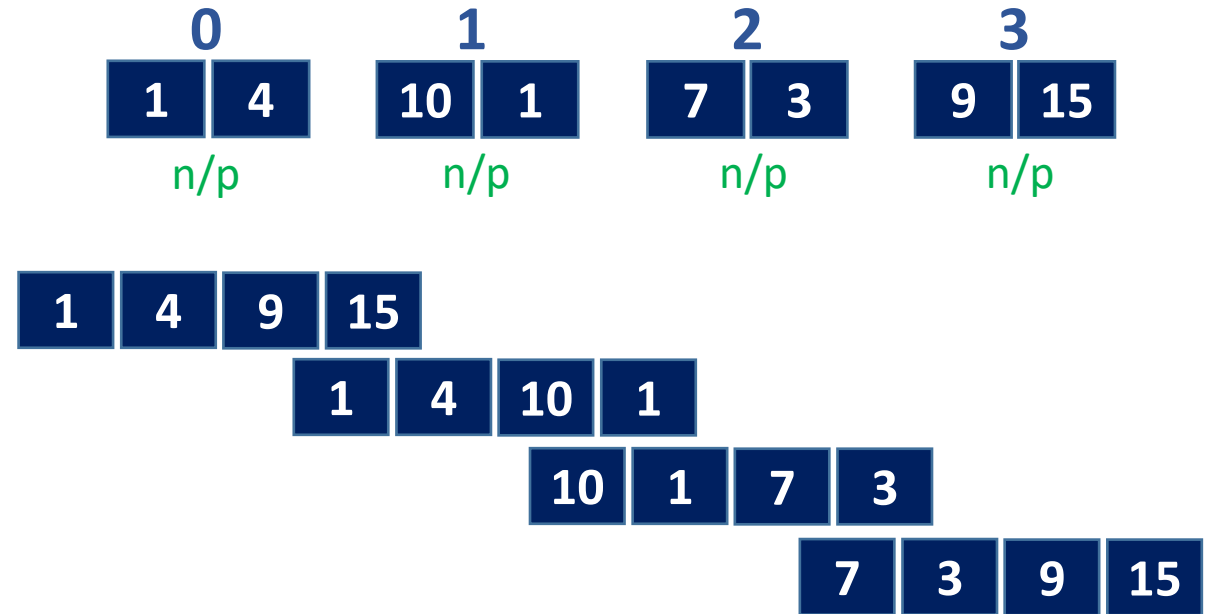
#1	0 -> 8 (2, $8n$)	
#2	0 -> 4 (1, $4n$)	8 -> 12 (1, $4n$)
#3	0 -> 2 (2, $2n$)	4 -> 6 (2, $2n$)
	8 -> 10 (2, $2n$)	12 -> 14 (2, $2n$)
#4	0 -> 1 (1, n)	8 -> 9 (1, n)
	2 -> 3 (1, n)	10 -> 11 (1, n)
	4 -> 5 (1, n)	12 -> 13 (1, n)
	6 -> 7 (1, n)	14 -> 15 (1, n)



$$\text{Total hop-bytes} = 2*8n + 1*4n + 2*2n + 1*n = 25n$$

Allgather – Ring Algorithm

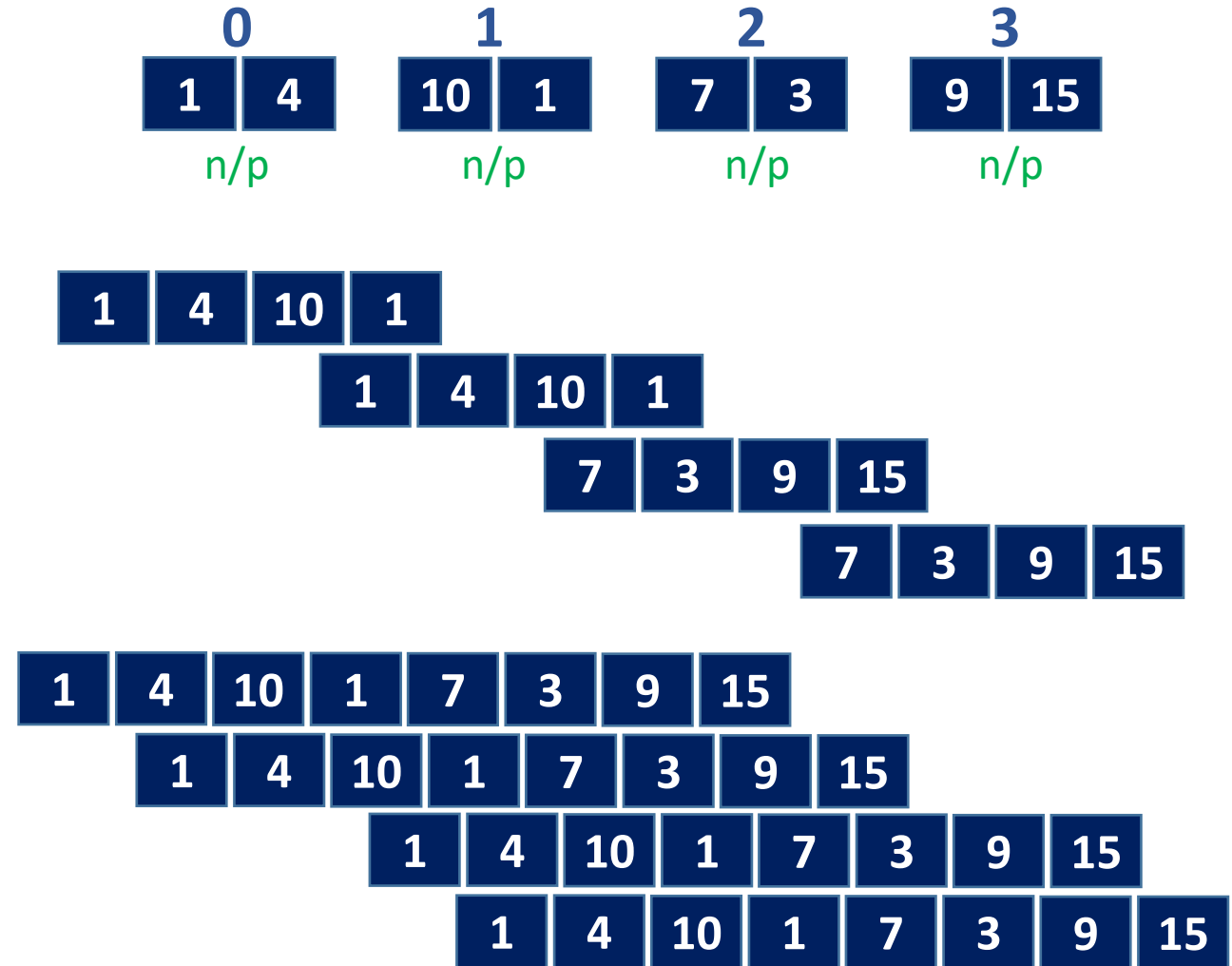
- Every process sends to and receives from everyone else
- Assume p processes and total n bytes
- Every process sends and receives n/p bytes
- Time
 - $(p - 1) * (L + n/p * (1/B))$
- Can we improve?



Similarly, $r \rightarrow (r+1)\%p$ data movement will occur for 2 more steps (total of $p-1$ steps)

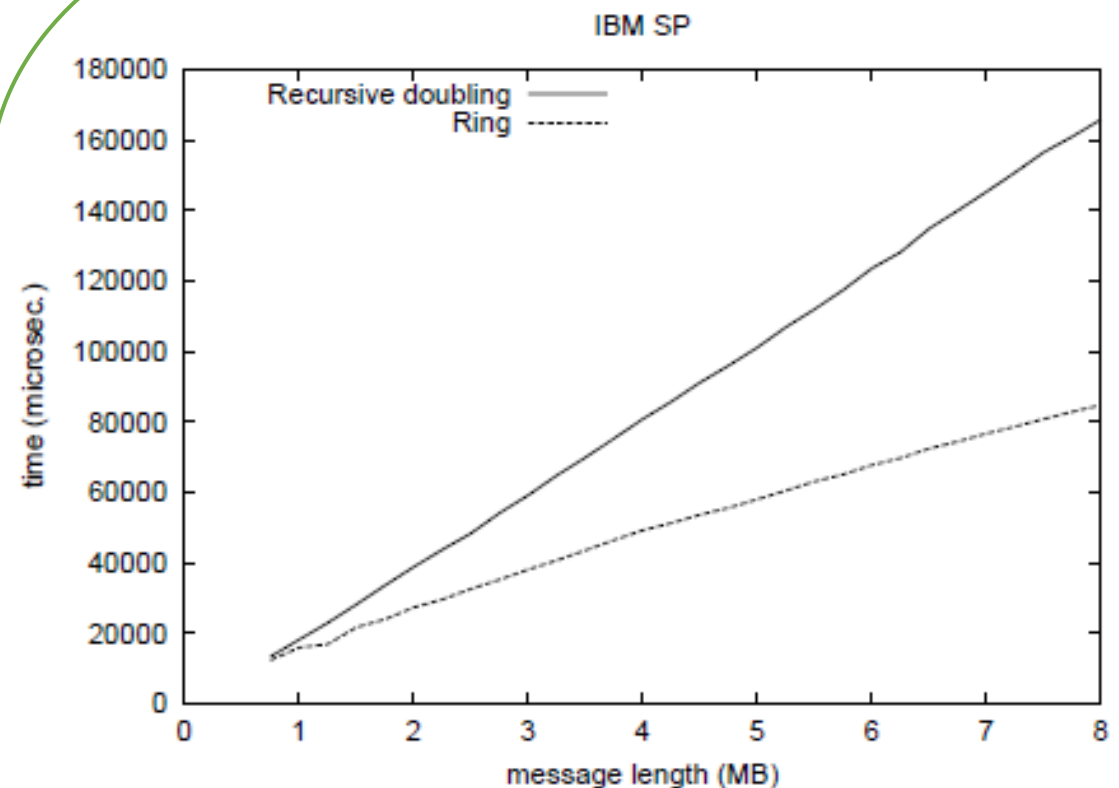
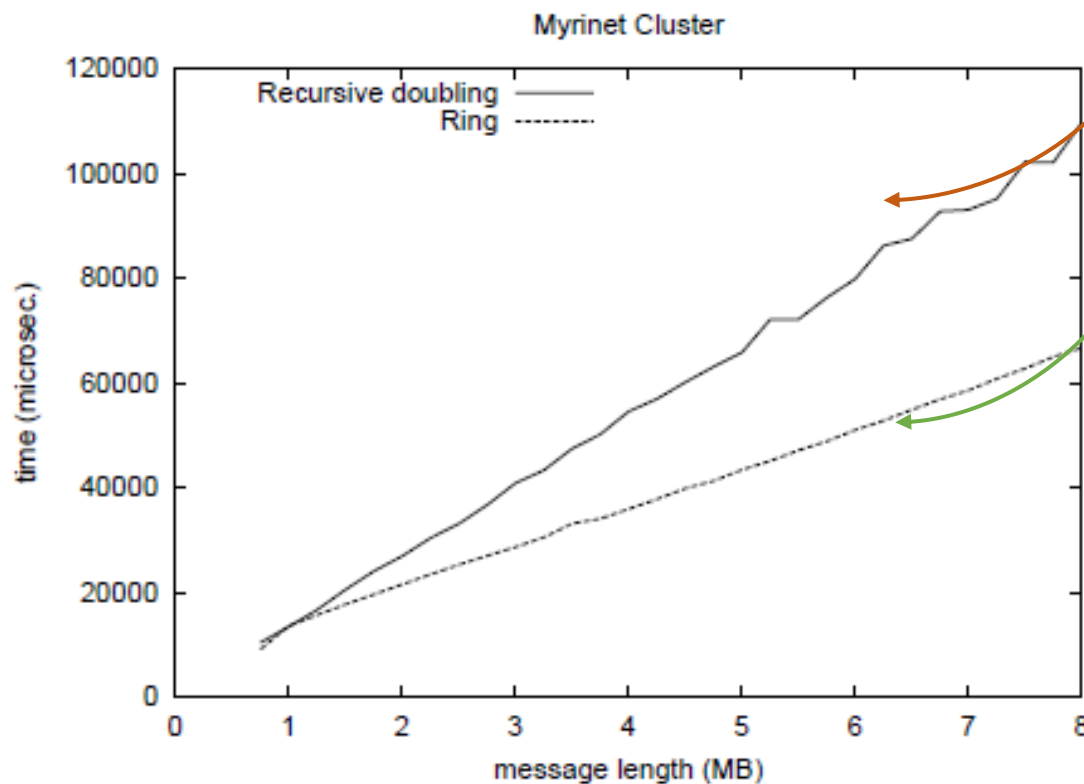
MPI_Allgather

- Every process sends and receives $(2^{k-1}) * n/p$ bytes at k^{th} step
- Time
- $(\log p) * L + (p-1) * n/p * (1/B)$
 - Latency term: $\log p$ steps
 - Bandwidth term: $n/p * [2^0 + 2^1 + 2^3 + \dots + 2^{(\log p) - 1}]$



Performance Comparison on 64 nodes

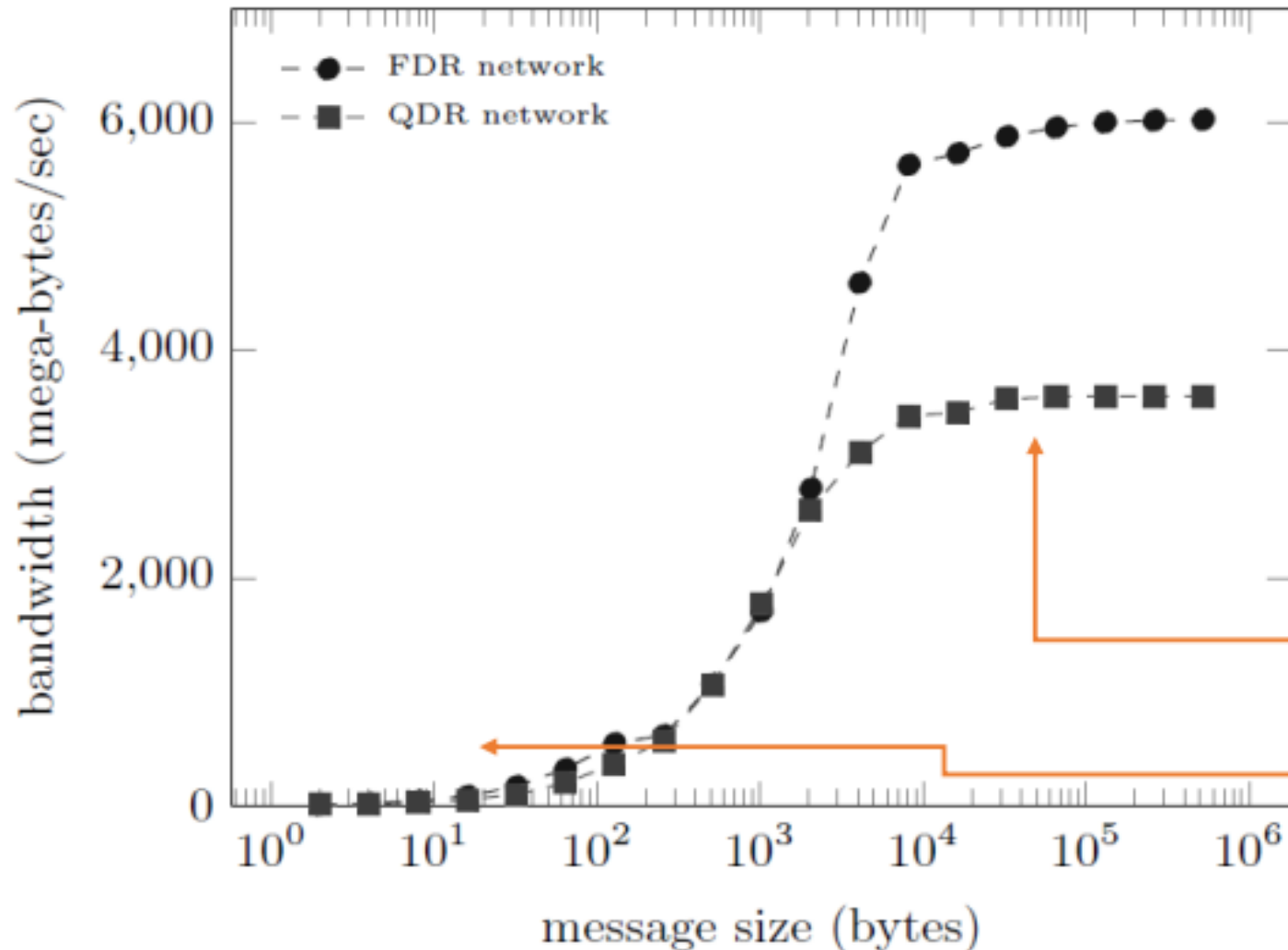
Why does ring perform better?



$$(\log p) * L + (p-1)*n/p*(1/B)$$

$$(p-1) * L + (p-1)*n/p*(1/B)$$

Recap (Latency and Bandwidth)



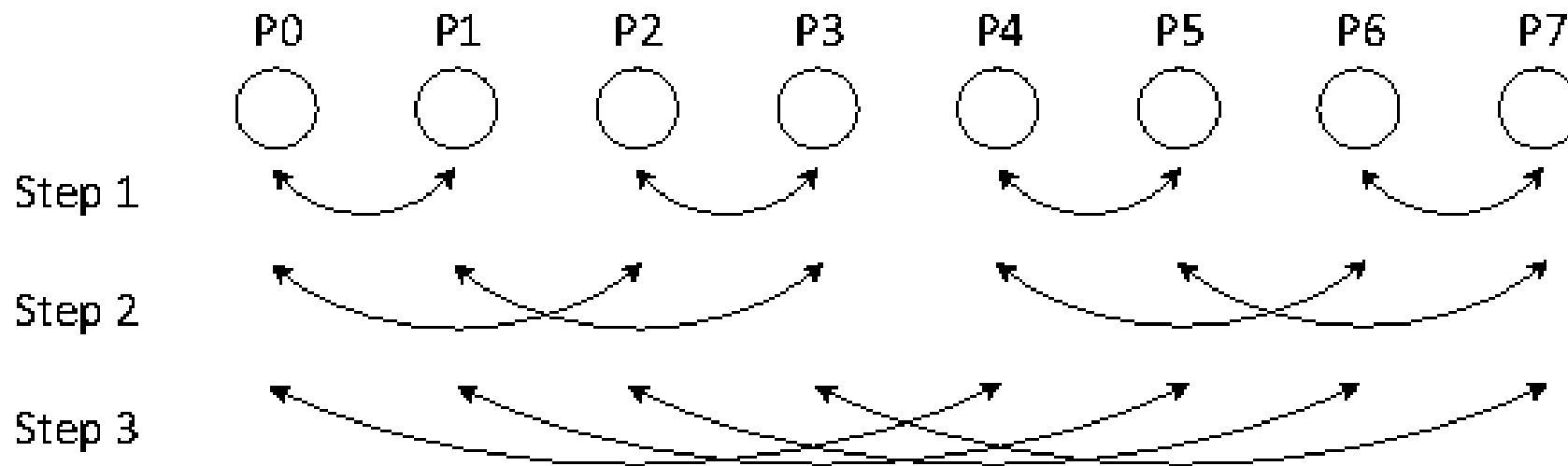
These numbers are typically derived from averaging the communication times of a constant set of communication pairs in the whole cluster/supercomputer.

Bandwidth-bound

Latency-bound

MPI_Allgather RD Illustration

Note: The higher the step number, the farther away are the communications (if sequential process mapping is assumed), and the bulkier are the communications.



H.W. Analyze hop-bytes on different networks and large process counts.

MPI_Allgather in MPICH

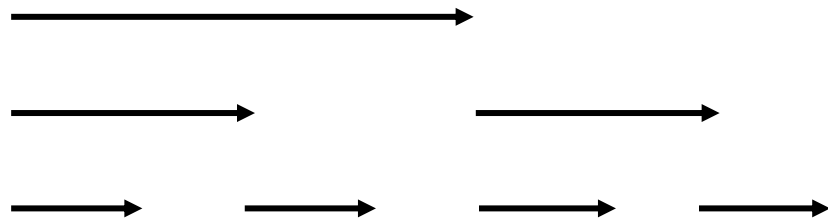
- `MPIR_CVAR_ALLGATHER_SHORT_MSG_SIZE=81920`
- `MPIR_CVAR_ALLGATHER_LONG_MSG_SIZE=524288`

- Bruck algorithm (*not covered here*)
 - short messages (< 80 KB) and non-power-of-two numbers of processes
- Recursive doubling
 - $(\log p) * L + (p-1) * n/p * (1/B)$
 - power-of-two numbers of processes and short or medium-sized messages (< 512 KB)
- Ring algorithm
 - $(p-1) * L + (p-1) * n/p * (1/B)$
 - long messages and any number of processes
 - medium-sized messages and non-power-of-two numbers of processes

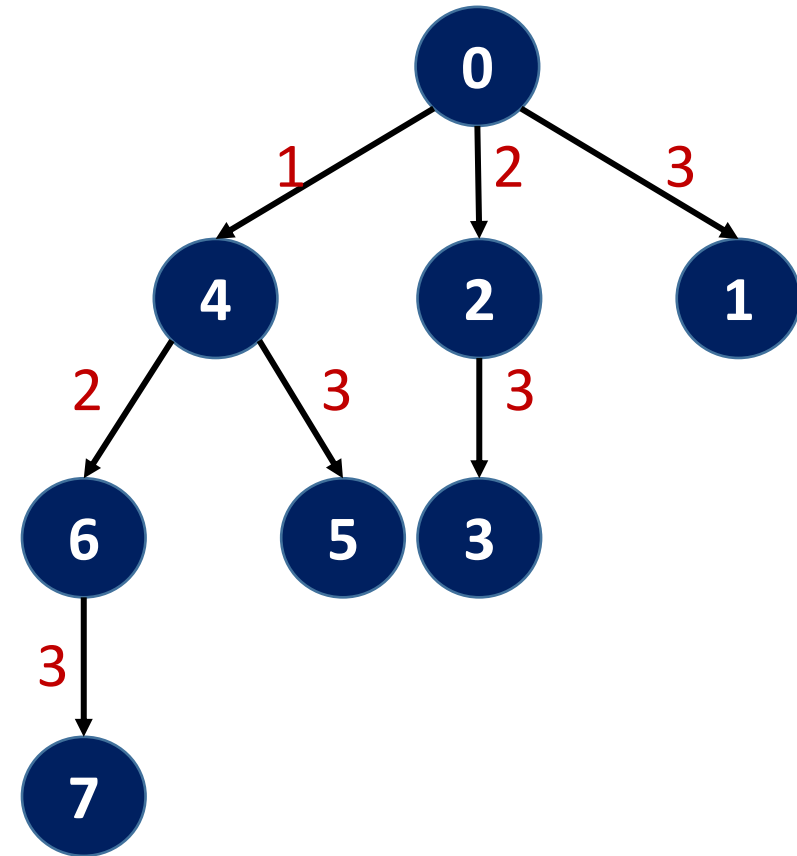
allgather.c

```
tot_bytes = (MPI_Aint) recvcnt * comm_size * type_size;
if ((tot_bytes < MPIR_CVAR_ALLGATHER_LONG_MSG_SIZE) && !(comm_size & (comm_size - 1))) {
    mpi_errno =
        MPIR_Allgather_intra_recursive_doubling(sendbuf, sendcount, sendtype, recvbuf,
                                                recvcnt, recvtype, comm_ptr, errflag);
} else if (tot_bytes < MPIR_CVAR_ALLGATHER_SHORT_MSG_SIZE) {
    mpi_errno =
        MPIR_Allgather_intra_brucks(sendbuf, sendcount, sendtype, recvbuf, recvcnt, recvtype,
                                    comm_ptr, errflag);
} else {
    mpi_errno =
        MPIR_Allgather_intra_ring(sendbuf, sendcount, sendtype, recvbuf, recvcnt, recvtype,
                                  comm_ptr, errflag);
}
```

Broadcast – Binomial Tree (Recap)



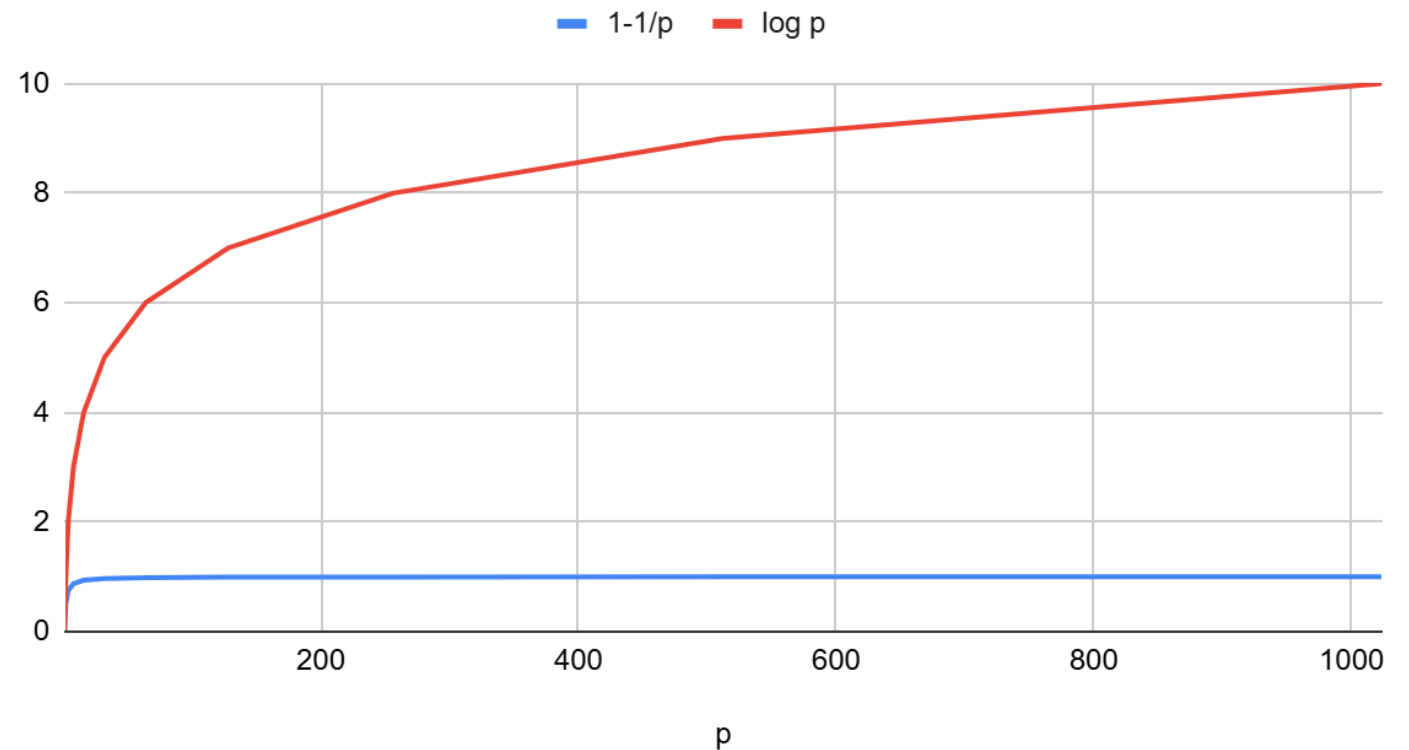
- #Steps for $p (=2^d)$ processes?
 - $\log p$
- Transfer time for n bytes
 - $T(p) = \log p * (L + n/B)$



$(1-1/p)$ vs. $\log p$

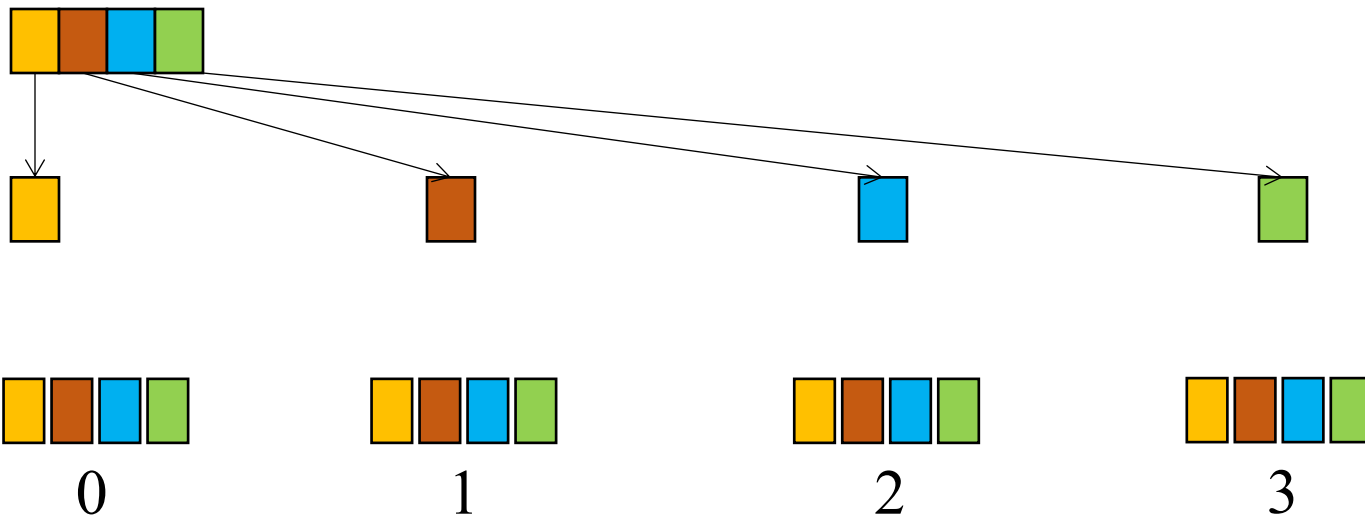
p	$1-1/p$	$\log p$
1	0	0
2	0.5	1
4	0.75	2
8	0.875	3
16	0.9375	4
32	0.96875	5
64	0.984375	6
128	0.9921875	7
256	0.99609375	8
512	0.998046875	9
1024	0.9990234375	10

1-1/p and log p



MPI_Bcast

- Message is first scattered from the root
- Scattered data is collected at all processes



MPI_Scatter

MPI_Allgather

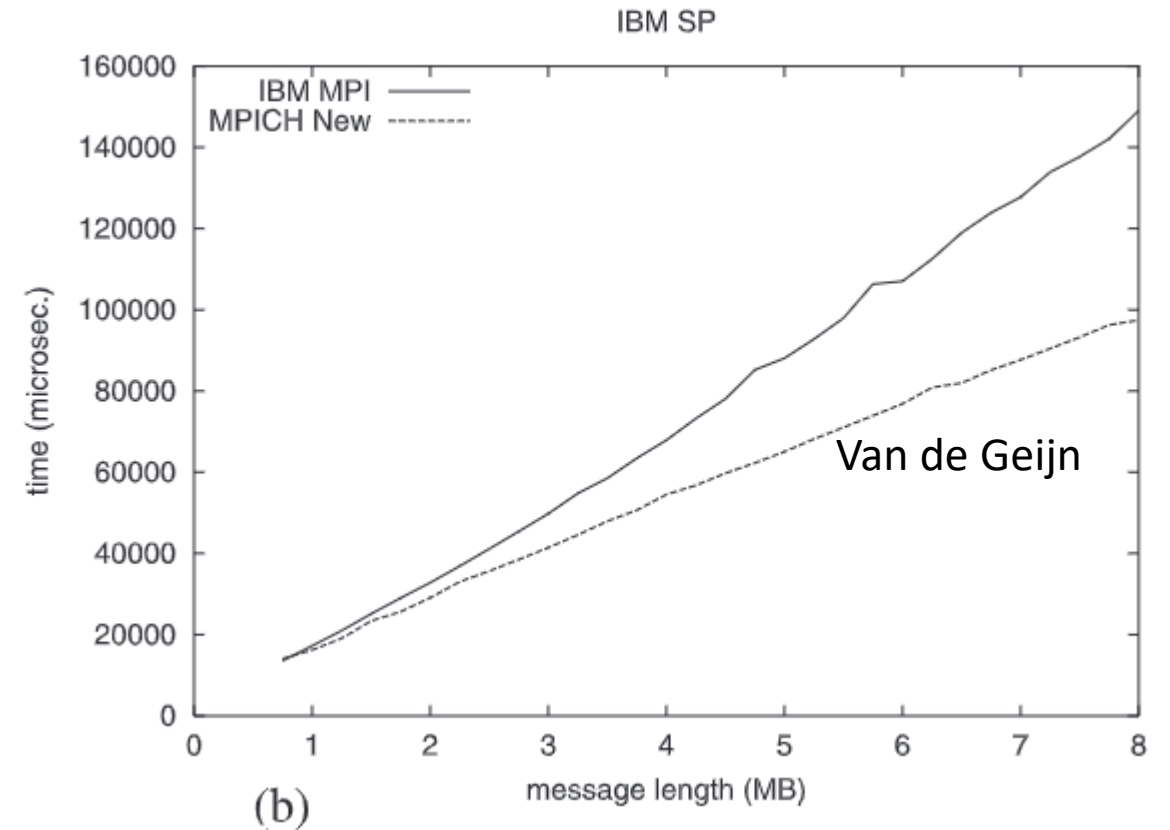
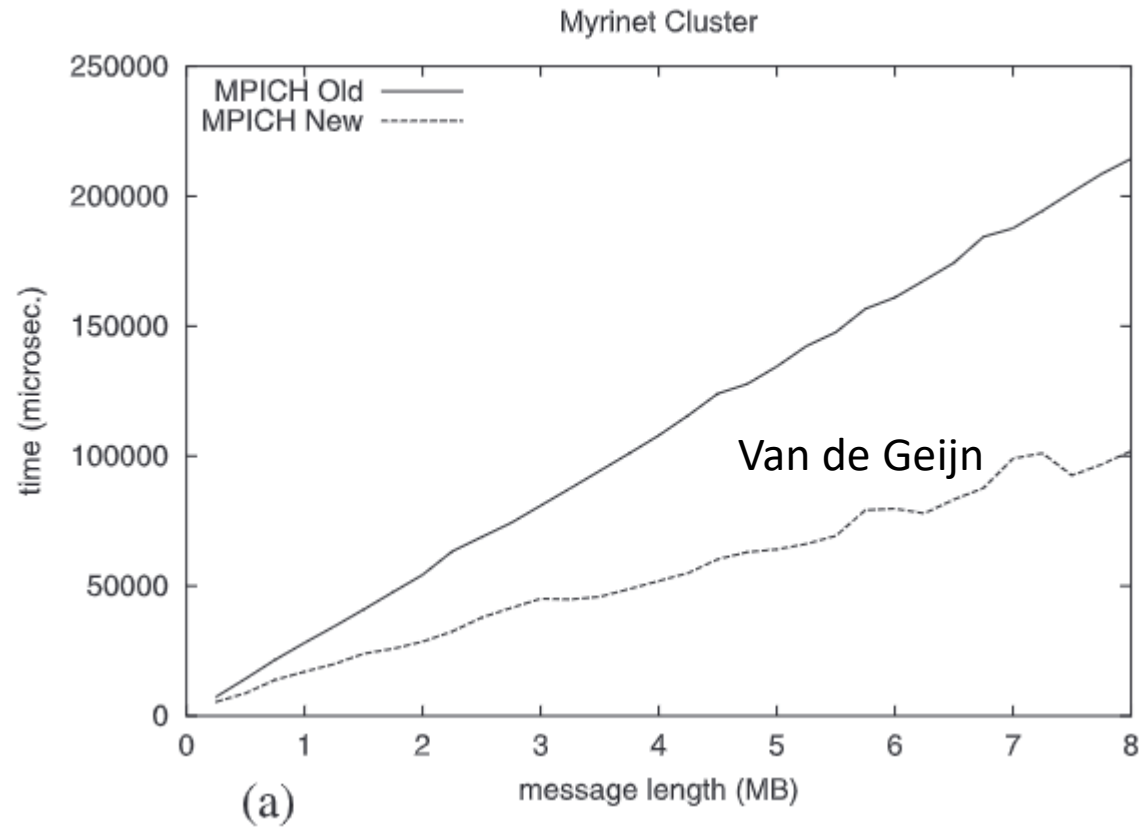
MPI_Bcast – Time Analysis

- **Time for broadcasting n bytes from root (binomial tree)**
 - $\log p * (L + n/B)$
- Time for scatter of n bytes from root
 - $\log p * L + (p-1)*(n/p)*(1/B)$
- Time for allgather (ring) of n/p bytes
 - $(p-1) * L + (p-1)*(n/p)*(1/B)$
- Time for allgather (recursive doubling) of n/p bytes
 - $(\log p) * L + (p-1)*(n/p)*(1/B)$
- **Time for broadcast of n bytes using scatter and allgather**
 - $(\log p + p-1) * L + 2((p-1)/p)*(n/B)$ OR
 - $(2 \log p) * L + 2((p-1)/p)*(n/B)$

Broadcast Algorithms in MPICH

- Short messages
 - $< \text{MPIR_CVAR_BCAST_SHORT_MSG_SIZE}$
 - Binomial
- Medium messages
 - Scatter + Allgather (Recursive doubling)
- Large messages
 - $> \text{MPIR_CVAR_BCAST_LONG_MSG_SIZE}$
 - Scatter + Allgather (Ring)

Old vs. New MPI_Bcast (64 nodes)



Communication Cost Model

- Message transfer time is modeled as $L+n/B$, where L is the latency (or startup time) per message, and $1/B$ is the transfer time per byte, and n the message size in bytes

Optimization of Collective Communication Operations in MPICH

Rajeev Thakur, Rolf Rabenseifner, William Gropp

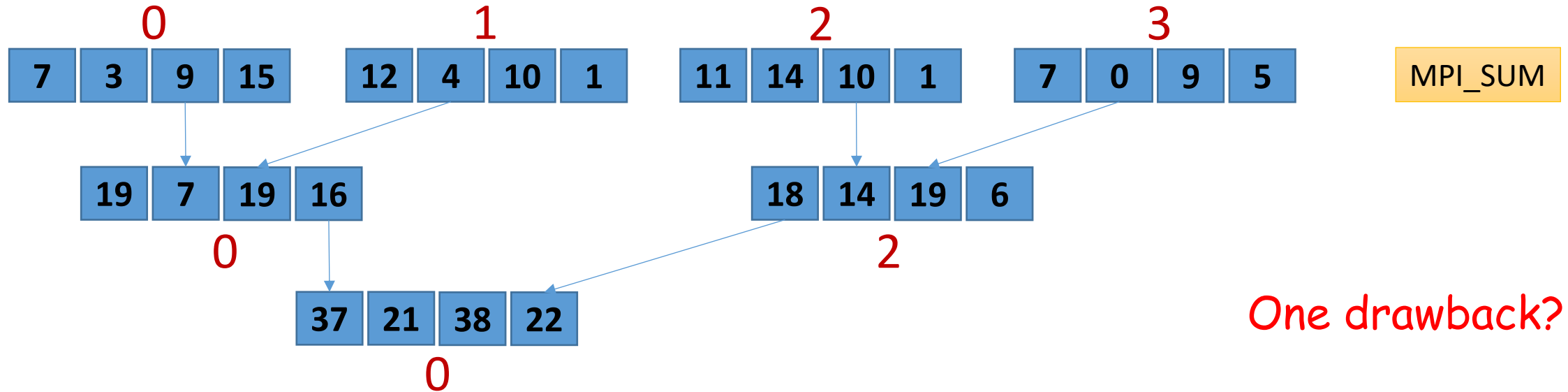
IJHPCA 2005

Optimization of Collective Reduction Operations

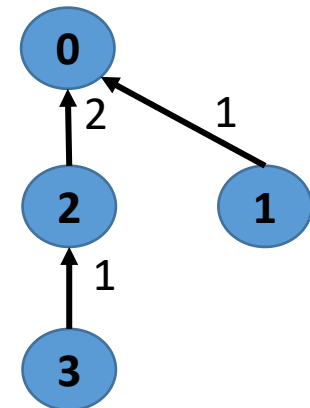
Rolf Rabenseifner

ICCS 2004

Reduce Algorithm – Recursive doubling (Recap)



One drawback?



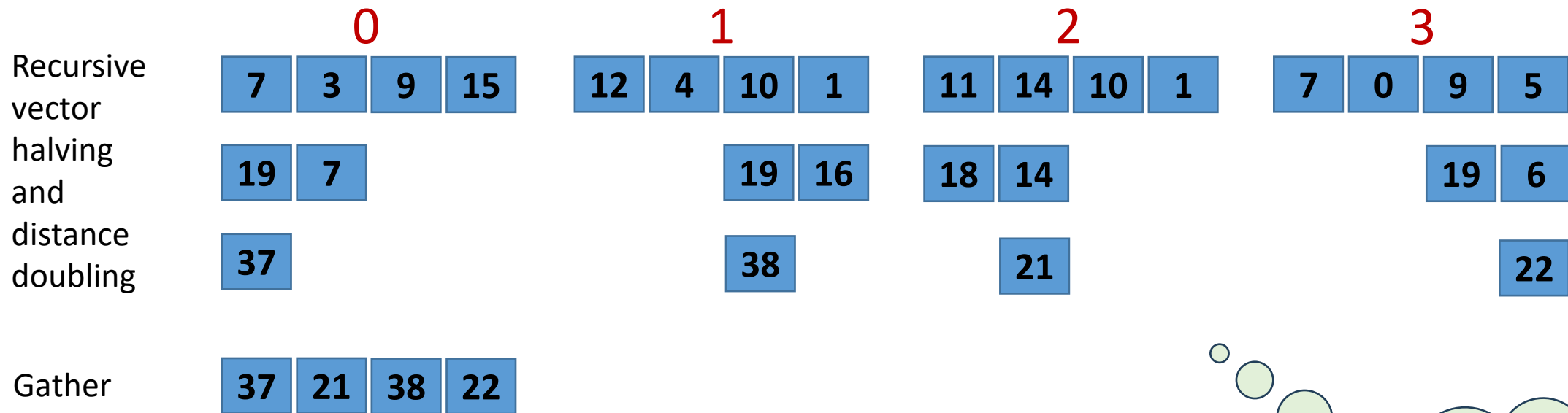
Time: $\log p (L + n \cdot (1/B) + n \cdot c)$

L = latency, B = bandwidth

c = compute cost per byte

Used for short messages

Reduce



Time:

$\log p * L + (p-1)/p * (n/B) + (p-1)/p * n * c$ (reduce-scatter) +
 $\log p * L + (p-1)/p * (n/B)$ (gather using binomial)

n = data size

L = latency

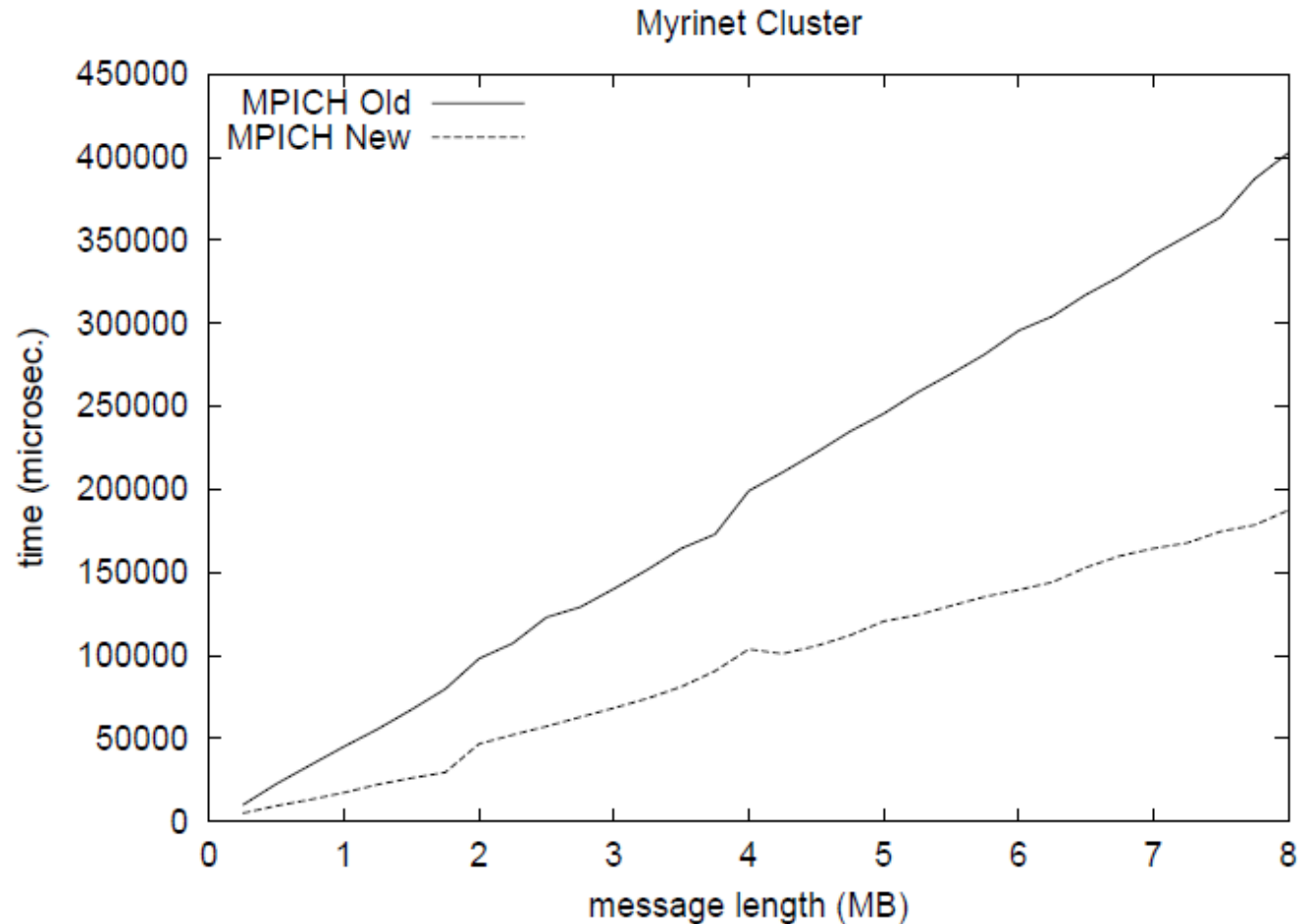
p = #processes

B = bandwidth

c = compute cost per byte

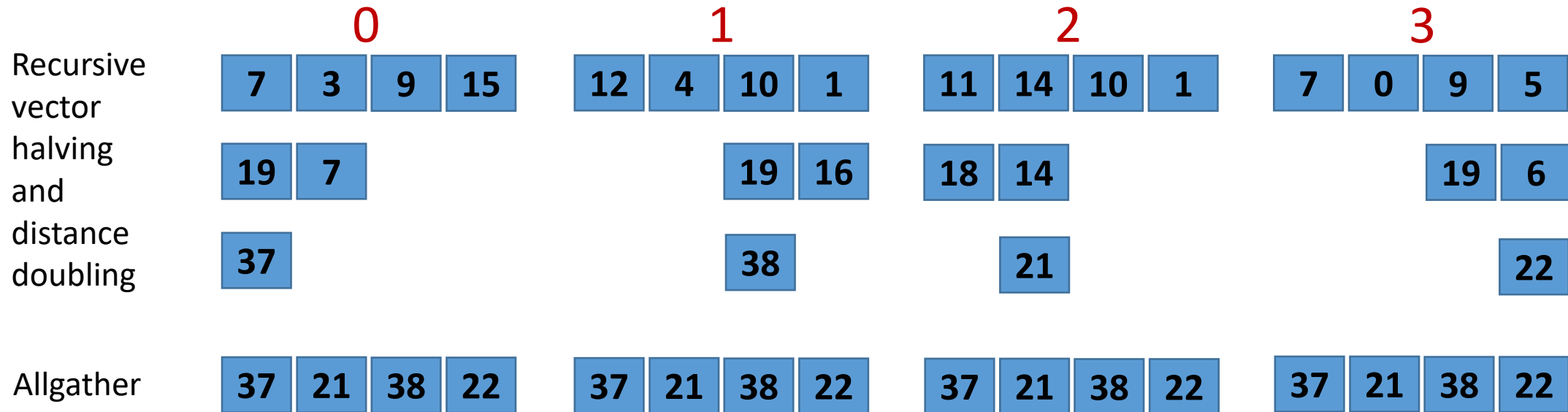
Do not let
any process
be idle!

MPI_Reduce on 64 nodes



Abstract. A 5-year-profiling in production mode at the University of Stuttgart has shown that more than 40% of the execution time of Message Passing Interface (MPI) routines is spent in the collective communication routines MPI_Allreduce and MPI_Reduce. Although MPI implementations are now available for about 10 years and all vendors are committed to this Message Passing Interface standard, the vendors' and publicly available reduction algorithms could be accelerated with new algorithms by a factor between 3 (IBM, sum) and 100 (Cray T3E, maxloc) for long vectors. This paper presents five algorithms optimized for different choices of vector size and number of processes. The focus is on bandwidth dominated protocols for power-of-two and non-power-of-two number of processes, optimizing the load balance in communication and computation.

Allreduce – Rabenseifner’s Algorithm



Time:

$\log p * L + (p-1)/p * (n/B) + (p-1)/p * n * c$ (reduce-scatter) +

$\log p * L + (p-1)/p * (n/B)$ (allgather using recursive vector doubling and distance halving)

n = data size

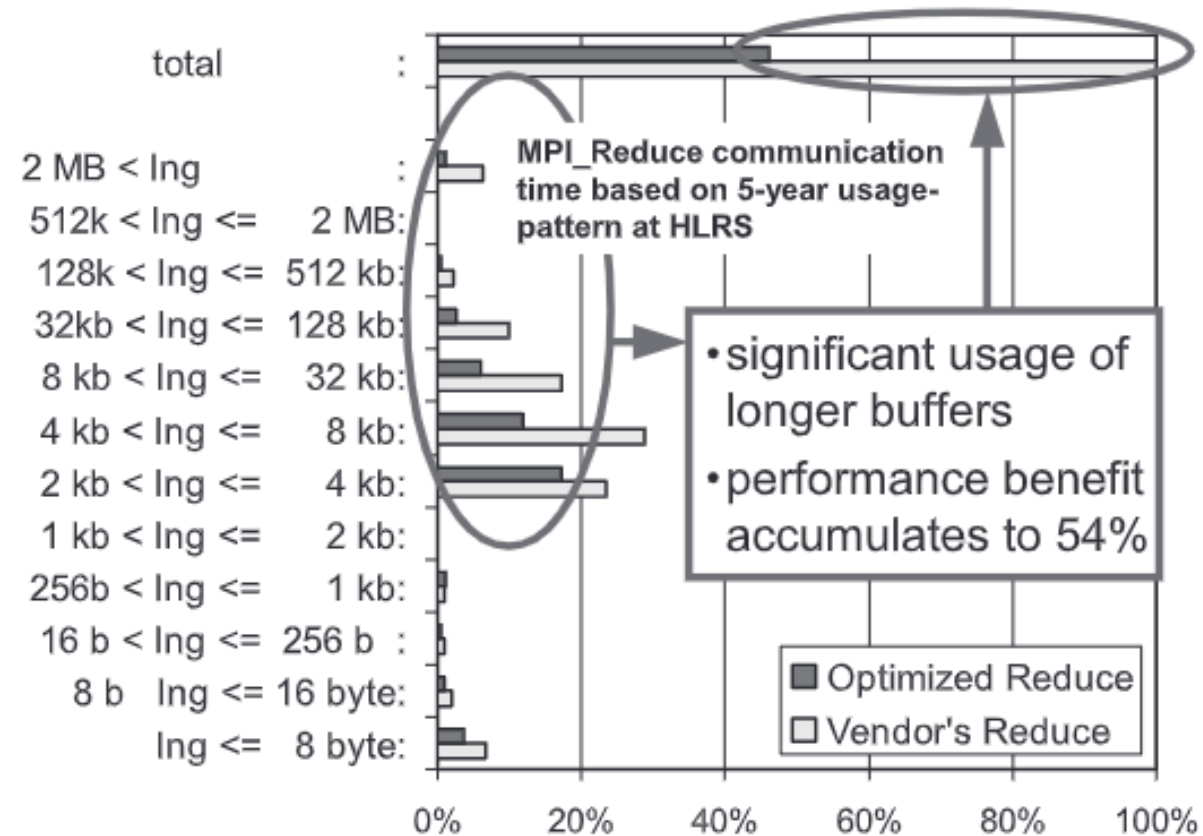
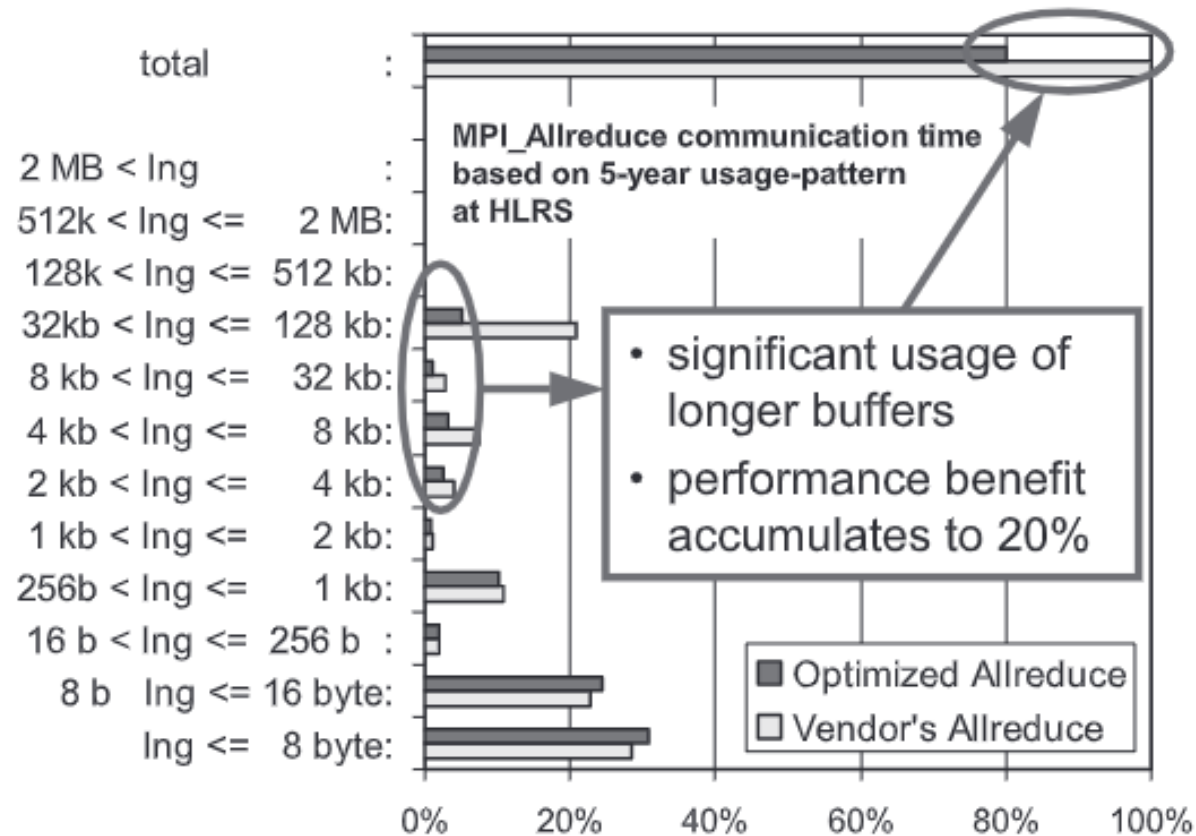
L = latency

p = #processes

B = bandwidth

c = compute cost per byte

Benefit for Long Messages



Allreduce Algorithms – Summary

Short - Medium messages

- Reduce (recursive doubling) followed by broadcast (binomial)
 - $[\log p (L + n \cdot (1/B) + n \cdot c)] + [\log p (L + n \cdot (1/B))]$

Long messages

- Reduce-scatter followed by allgather (recursive doubling)
 - $2 \cdot \log p \cdot L + 2(p-1)/p \cdot (n/B) + (p-1)/p \cdot n \cdot c$